

206. Reverse Linked List

PLATFORM

Platform: LeetCode

DIFFICULTY

EASY

DATE

Solved: June 23, 2026

Problem Summary

Given the head of a singly linked list, reverse the list and return the new head.

Key Insights

- Reversal happens **in-place** — no extra list/array needed.
- You must save `curr.next` *before* overwriting it, or you lose the rest of the list.
- `prev` ends up as the new head once `curr` becomes null.

Section

Time Complexity : $O(n)$ — one pass through all nodes.

Space Complexity : $O(1)$ — only three pointers used, no extra data structures.

Approach

Iterative pointer reversal — walk through the list once, and at each node, flip its next pointer to point backward instead of forward, using a `prev` pointer to track the new previous node and a `next` pointer to save the original next node before overwriting it.

Observations

- Classic three-pointer technique (`prev`, `curr`, `next`) — appears in many linked list problems.
- No dummy node needed here since we're not modifying structure around a head boundary, just direction.
- Single pass, no recursion stack used (unlike the recursive variant of this problem).

Algorithm

- Initialize `prev = null`, `curr = head`.
- While `curr != null`:
 - a. Store `next = curr.next`.
 - b. Reverse link: `curr.next = prev`.
 - c. Move `prev = curr`.
 - d. Move `curr = next`.
- Return `prev` (new head).

Points to Remember

- **Three-pointer reversal pattern:** `prev`, `curr`, `next` — reusable for reversing sublists (e.g., reverse between positions, reverse in groups of k).
- Always cache `curr.next` before reassigning `curr.next`, or the chain breaks.
- The loop's exit condition (`curr == null`) means `prev` naturally holds the final node — that's your new head/tail depending on direction.
- This pattern generalizes to "reverse a portion of a list" by just controlling where you start/stop the loop and reconnecting boundaries afterward.

Linked List

DescriptionEditorialSolutionsSubmissions

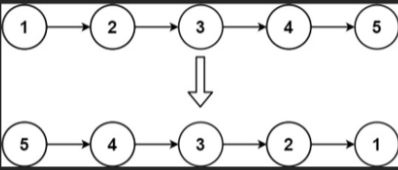
206. Reverse Linked List

Solved

EasyTopicsCompanies

Given the head of a singly linked list, reverse the list, and return the reversed list.

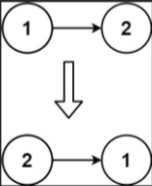
Example 1:



```
graph LR; 1((1)) --> 2((2)); 2 --> 3((3)); 3 --> 4((4)); 4 --> 5((5)); 5 --> null; 5 --> 4; 4 --> 3; 3 --> 2; 2 --> 1; 1 --> null;
```

Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]

Example 2:



```
graph LR; 1((1)) --> 2((2)); 2 --> null; 2 --> 1; 1 --> null;
```

Input: head = [1,2]
Output: [2,1]

Example 3:

24.7K422301 Online

Accepted

28 / 28 testcases passed

Sahil Khan submitted at Jun 23, 2026 21:25

Unlock the Full LeetCode Experience

Runtime

0 ms | Beats 100.00%

Memory

43.98 MB | Beats 91.98%

Code | Java

```
1 /**
2  * Definition for singly-linked list.
3  * public class ListNode {
4  *     int val;
5  *     ListNode next;
6  *     ListNode() {}
7  *     ListNode(int val) { this.val = val; }
8  *     ListNode(int val, ListNode next) { this.val =
9  * }
```

View more

Code

Java

```
5 *     ListNode next;
6 *     ListNode() {}
7 *     ListNode(int val) { this.val = val; }
8 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
9 * }
10 */
11 class Solution {
12     public ListNode reverseList(ListNode head) {
13         ListNode prev = null;
14         ListNode curr = head;
15         ListNode next;
16
17         while(curr != null){
18             next = curr.next;
19             curr.next = prev;
20             prev = curr;
21             curr = next;
22         }
23
24         return prev;
25     }
26 }
```

Saved

Ln 20, Col 25

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1Case 2Case 3

Input

head = [1,2,3,4,5]

Output